

Cerca · App móvil · Sprint 2 · Publicar, reservar, reseñar y entregar

Sprint final. Jorge y Salvador. Dos semanas. Al cerrar, la app se instala desde un enlace en un Android y en un iPhone que no son nuestros, y quien la abre busca, publica, reserva y reseña sin que se rompa.

De dónde partimos

El sprint 1 dejó cerrado: el monorepo con `@cerca/contract`, la regla de dependencia en el linter, el cliente HTTP que valida con `parse` en el límite, la sesión persistente con llavero y refresh, la capa de datos con claves jerárquicas y `snapToGrid`, la búsqueda con sus cuatro estados, `expo-location` con degradación y selector de ciudad, el tema semántico, `i18next`, la tarjeta de resultado y `verify.sh` corriendo en CI.

Historias cerradas: **US-01**, **US-02**, **US-08** y la mitad de **US-07**.

Falta todo el lado de escritura del producto y la entrega. Es este sprint, y no hay un tercero.

Historia	Qué falta	Responsable
US-03	Publicar un anuncio en 4 pasos	Jorge
US-04	Editar solo mis anuncios	Jorge
US-05	Solicitar una reserva	Salvador
US-06	Reseñar una vez, dentro de plazo	Salvador
US-07	Precio completo y distancia en detalle y reservas	Salvador
US-09	Retirar un anuncio denunciado	Jorge
US-10	Instalar una build de QA desde un enlace	Jorge

Qué se demuestra al cerrar el sprint

El guion de la demo, con un teléfono que no es nuestro y sin tocar el portátil:

1. Abro el enlace, instalo la app, entro. Cierro y reabro: sigo dentro.
2. Busco, abro un anuncio. El precio se lee `$450 / hora · mínimo 2 h`, la valoración `4,8 · 200 reseñas`, la distancia `a 3 km`. Lo comparto por WhatsApp y el enlace abre ese anuncio.
3. Me hago proveedor desde la app. Aparece "Mis anuncios" sin reiniciar nada.
4. Publico un anuncio en cuatro pasos: "por hora" me pide horas mínimas, "presupuesto" no me pide precio, subo dos fotos, publico.
5. Con otra cuenta reservo ese servicio. Pulso dos veces y solo hay una reserva. Vuelvo atrás y el estado ya es el nuevo.
6. El proveedor acepta y completa. El cliente reseña. Intento reseñar otra vez y el botón está deshabilitado con "Ya reseñaste esta reserva". Con una reserva de hace 40 días, el mensaje es otro.
7. Abro el anuncio de otra persona: no hay botón de editar. Lo denuncio, y con la cuenta de moderador lo retiro.

8. Pongo el teléfono en alemán y la fuente al 200%: nada se corta, el precio se lee `1.299,90 MX$` y la distancia sigue en kilómetros.
9. Modo avión a mitad de una reserva: mensaje claro y no se pierde nada.

Día cero · los dos juntos, medio día, antes de escribir una línea

Tres cosas bloquean y ninguna se arregla sola el último día.

1 · pricing e images[] en el resultado de búsqueda. Es el punto 3 de `contract-delta.md` y sigue abierto. Sin `pricing`, la tarjeta no puede escribir "\$450 / hora": no sabe si esos 450 son la hora o el trabajo entero. Sin `images[]` no hay foto que virtualizar. Es una línea en `toSearchItem` del backend y el dato ya está en la fila. Se pide el día cero con fecha. **Si el día 3 no está**, la tarjeta se queda con "Desde \$450", el criterio de US-07 se demuestra en el detalle —que sí trae `pricing`— y se anota en el delta. El dato no se inventa en el cliente.

2 · Los endpoints de escritura, con Postman delante. `POST /listings`, `PATCH /listings/:id`, `photos:presign`, `POST /bookings`, `/accept`, `/decline`, `/complete`, `/cancel`, `/review`, `/reports`, `/moderate`. De cada uno hace falta la forma exacta del 403 y del 409: qué `code` y qué `reason` mandan. Los motivos de la política de reseña (`not_your_booking`, `not_completed`, `already_reviewed`, `window_closed`) tienen que coincidir **letra por letra** con los del servidor, porque son la misma clave de i18n en los dos lados. Si no coinciden, el usuario ve "Algo ha salido mal" donde debería leer la regla.

3 · El entorno de Android y las credenciales de tienda. Android Studio, el SDK, el JDK y un teléfono de gama media instalados y funcionando el día cero, no el día que toque medir. Android necesita además un keystore, y lo genera EAS. iOS necesita una cuenta de Apple Developer de pago y, o bien los UDID de los iPhone donde se va a instalar (ad-hoc), o bien TestFlight. Sin resolverlo el día cero, **US-10 no cierra en iOS** y no hay forma de arreglarlo en la última tarde.

Lo que **no** entra, aunque esté en el delta: generar los tipos de la app desde el OpenAPI del backend. Es buena idea y son dos días en un sprint que ya está lleno. Se mantiene el espejo a mano, y el riesgo se asume por escrito.

Entorno · se trabaja con Android Studio

El sprint 1 se hizo contra iOS. Este se hace en Android, y no es una preferencia: **cuatro criterios del DoD no se pueden demostrar sin las herramientas de Android** —los 55 FPS con 5.000 tarjetas, la memoria estable en un scroll largo, la fuente al 200% y la instalación desde un enlace en un teléfono físico—. El emulador y el portátil mienten los dos; el profiler, no.

El proyecto ya está listo: `ios/` y `android/` están en `.gitignore` y no hay un solo archivo nativo versionado. La carpeta nativa se **genera**, no se guarda.

Lo que hay que instalar, una vez y los dos:

- Android Studio, que trae el SDK, el emulador y `adb`.
- Un JDK 17. Vale el que Android Studio incluye (`jbr`) apuntando `JAVA_HOME`.
- `ANDROID_HOME` y las `platform-tools` en el `PATH`, en `~/.zshrc`.
- Un emulador de gama media y **un Android físico de gama media**. Las medidas de rendimiento del DoD solo cuentan en el físico.

Y después, desde la raíz del repo:

- `npm run -w mobile android` genera `android/`, compila e instala la development build.
- Android Studio se abre sobre `apps/mobile/android`, ya generada.

Dentro de `android/` no se edita nada. Es carpeta generada: el siguiente `expo prebuild --clean` se lleva por delante lo que haya. Todo cambio nativo —permisos, manifest, versiones de SDK— entra por `app.json` o por un plugin, y se comprueba regenerando.

Dos trampas que van a costar una tarde si no están escritas:

- **localhost no existe para el emulador:** apunta al propio Android. El backend de la máquina es `http://10.0.2.2:3333/v1`, y en un teléfono físico, la IP de la máquina en la red local. Va en `.env.local`, y se anota en `.env.example`.
- **Android bloquea el HTTP en claro fuera de debug.** La development build funciona, pero el APK de preview contra un backend `http://` falla sin decir por qué. Se resuelve con `expo-build-properties` (`usesCleartextTraffic` en el perfil de preview) o con un backend en HTTPS. Es de Jorge y va en J6.

Lo que Android Studio aporta al sprint, y por lo que se abre: **Logcat** para el crash nativo que Metro no enseña, el **Profiler** de memoria y CPU para el scroll largo, y el **Layout Inspector** para la jerarquía de la tarjeta. El día a día sigue siendo VS Code y Metro; Android Studio no sustituye el ciclo de Expo.

iOS no se abandona: la app se entrega en las dos plataformas (US-10) y se sigue probando en el iPhone antes de cada PR. Solo cambia dónde se mide y dónde se depura.

Reparto

El corte es por verticales completas, igual que en el sprint 1: cada uno se lleva pantallas de arriba abajo con sus schemas, sus hooks y sus textos, y no se pisan archivos.

Bloque	Jorge	Salvador
Escritura	Publicar, editar, mis anuncios	Reservar, gestionar reservas, reseñar
Autoridad	Kit de autorización de UI, capacidad de proveedor	La política de reseña y sus motivos
Producto	Moderación	Detalle de anuncio, favorito, compartir
Cierre	EAS: Android e iOS instalables	i18n, accesibilidad y rendimiento de lo nuevo

Reglas del reparto

- **Jorge entrega el kit compartido el día 2, en `main`.** Salvador depende de él para las reservas. Si el día 2 no está, es un bloqueo y se dice en voz alta, no se duplica.
- El contrato se toca en **archivos nuevos y separados** (`listing-write.ts` de Jorge, `booking.ts` y `review.policy.ts` de Salvador). Los archivos del sprint 1 no se editan sin avisar al otro.
- Ninguna comprobación de permiso del cliente protege nada. Todas las que se escriben aquí sirven para no enseñar un botón que va a devolver 403.

Jorge

J0 · Kit compartido · día 1 y 2, bloquea a Salvador

Lo que las dos columnas necesitan, en un solo sitio y antes que nada.

- `useCan(permission)`, `<Can permission>` y `withCapacity('provider')` en `presentation/auth/`. Reflejan `can()` y `has()` del contrato sobre el `Actor` de la sesión. Son UX, nunca seguridad.
- La regla de los dos comportamientos, aplicada en el propio kit: **sin capacidad, el control no se pinta; bloqueado por política, se pinta deshabilitado y con su motivo.**
- Cabecera `Idempotency-Key` en `HttpRequest`, con `randomUUID()` de `expo-crypto`. Una clave por intento del usuario, no por render.
- `problemMessageKey(error)` en `presentation/i18n/`: de `problem.code` y `problem.reason` a clave de `i18n`, en un solo mapa. Hoy cada pantalla traduce el error a su manera.

J1 · Capacidad de proveedor y navegación

- `POST /me/capacities/provider` y refresco del `Actor`: al volver, "Mis anuncios" aparece sin reiniciar la app.
- `(app)/_layout.tsx` pasa de `Stack` a `Tabs`.
- El grupo de proveedor cuelga de `(app) (src/app/(app)/(provider)/)` para que la pestaña se pueda ocultar con `href: null` cuando la cuenta no tiene la capacidad. La guarda sigue viviendo en un único `_layout`.

J2 · Mis anuncios

- `GET /me/listings` con los cinco estados y su badge con texto.
- `POST /listings/:id/publish` y `/pause`, optimistas con `rollback`, invalidando `listingKeys.mine()` y `listingKeys.searches()`.
- Los cuatro estados de la pantalla, con el vacío inicial llevando a "publicar el primero".

J3 · Publicar en 4 pasos · US-03

- React Hook Form multipaso con **un solo schema** de `Zod` y `zodResolver`. Cada paso valida su trozo; nada se revalida a mano.
- La unión `Pricing` dirige el paso 2: `hourly` pide `minimumHours`, `quote` **no pide precio ni lo tiene en el tipo**, `fixed` pide un importe. Un `switch` con `assertNever` sobre `model`.
- El anuncio se crea como `draft` al terminar el paso 3, y las fotos se suben al paso 4 contra `POST /listings/:id/photos:presign` + `PUT` a `uploadUrl`. No es un capricho: el `presign` necesita un `id`. Efecto de lado bueno: **el borrador retomable es el draft del servidor**, no un JSON en el teléfono.
- Las fotos se reducen con `expo-image-manipulator` **antes** de subir. Una foto de 4000×3000 son ~48 MB descomprimidos y el móvil de gama media se cae.
- `POST /listings/:id/publish` cierra el flujo.

J4 · Editar solo lo mío · US-04

- `PATCH /listings/:id` reutilizando el formulario del paso correspondiente.
- `canEditListing(actor, listing)` en el contrato, devolviendo **motivo** y no booleano.
- En anuncio ajeno el botón **no existe**. En anuncio propio retirado, existe deshabilitado y con su motivo.

- Si se fuerza la petición, el 403 con `not_owner` se traduce y se enseña. La app no pretende impedirlo.

J5 · Moderación · US-09

- Grupo `(moderation)` guardado por `can(actor, 'report:resolve')`, pestaña oculta para todos los demás.
- `GET /reports` con los cuatro estados, `POST /reports/:id/resolve`.
- `POST /listings/:id/moderate` para `under_review` y `removed`, con motivo.
- `POST /reviews/:id/moderate`, deshabilitado con motivo cuando el moderador es el autor de la reseña.

J6 · Entrega · US-10

- `eas.json` con `development`, `preview` y `production`.
- **Android:** perfil `preview` con `buildType: apk` y distribución interna. Enlace y QR.
- `expo-build-properties` para el HTTP en claro del perfil de preview, o backend en HTTPS. Sin esto el APK instala, abre y **no carga nada**, sin error visible.
- **iOS:** distribución interna ad-hoc con los UDID registrados, o TestFlight. Lo que se decidiera el día cero.
- `expo-updates` con canal `preview` para mandar JS, estilos y traducciones sin rebuild.
- `app.json` al día: `version`, `runtimeVersion`, iconos, splash, y los textos de permiso de **ubicación y fotos**, en los dos idiomas.
- Alguien que no escribió la app la instala en un Android y en un iPhone **físicos** y la abre. Dos líneas en el README con los pasos.

Salvador

S1 · Detalle de anuncio, favorito y compartir

- Ruta `(app)/listings/[id].tsx` con `GET /listings/:id`, sus cuatro estados y el caso "retirado o no existe", que no es un error de red.
- Deep link `cerca://listing/:id` y botón de compartir con `Share`. Compartir un anuncio es una función central del producto.
- **Aquí se cierra US-07:** `pricing` completo con la unión discriminada, `$450 / hora · mínimo 2 h`, valoración con recuento y plural, distancia en km o millas.
- Favorito con **mutación optimista**: `cancelQueries`, `snapshot`, `rollback` en `onError`, y en `onSettled` invalidar `detail(id)` y `searches()`. El corazón también se pinta en la lista.
- `POST /listings/:id/report` desde el detalle.
- `expo-image` con `cachePolicy: "memory-disk"`, `blurhash` y `contentFit`.

S2 · Reservar · US-05

- `POST /bookings` con `Idempotency-Key`: una clave por intento del usuario. Pulsar dos veces crea **una** reserva.
- Botón deshabilitado mientras la petición vuela, con su texto de "enviando".
- No se puede reservar el anuncio propio: control deshabilitado con motivo, no ausente.
- Al volver atrás, la lista y el detalle ya muestran el estado nuevo.
- Modo avión a mitad: mensaje claro y no se pierde lo escrito.

S3 · Mis reservas y sus estados

- `GET /bookings?role=customer|provider` en una pantalla con selector de lado. El lado de proveedor no se ofrece si la cuenta no tiene la capacidad.
- Detalle con `switch` sobre `status.kind` y `default: assertNever(status)`: el día que el backend añada una variante, la app **deja de compilar** señalando el sitio exacto.
- Acciones según el lado: cliente cancela; proveedor acepta, rechaza y completa. Cada una con su optimismo y su invalidación.
- Los cuatro estados en la lista.

S4 · Reseñar · US-06 · la función estrella

- `canReviewBooking(actor, booking, now)` en `packages/contract/src/review/review.policy.ts`. Pura, con `now` como parámetro, y devolviendo `{ ok: false, reason }`, nunca un booleano.
- Los cuatro motivos son claves de i18n: la pantalla hace `t('review.blocked.' + reason)` y el dominio no sabe en qué idioma se enseña.
- Tests: los cuatro motivos, el camino feliz, y el borde exacto de los 30 días (día 30 sí, día 31 no). Sin simular relojes.
- `POST /bookings/:id/review` con `Idempotency-Key`. El 409 del servidor se traduce con la **misma clave** que la política del cliente.
- El botón bloqueado se **deshabilita y explica**. Nunca desaparece: un botón que se esfuma deja al usuario preguntándose qué pasó.
- `GET /listings/:id/reviews` en el detalle, con plural correcto en 0, 1 y 2.

S5 · i18n y accesibilidad de todo lo nuevo

- `en.json` y `es.json` completos para las pantallas nuevas, con plurales `_one / _other`, interpolación y claves tipadas.
- `useAnnounceFirstError()` en `presentation/all/`, entregado el día 4 para que Jorge lo use en el formulario de publicación.
- Alemán: revisión de layout en formulario de publicación, reserva y reseña, que son los que revientan.
- Una tarjeta = una parada del lector en las tres listas nuevas (reservas, reseñas, cola de moderación).
- Fuente al 200% sin cortes. Ningún estado comunicado solo por color. Área tátil $\geq 44 \times 44$.

S6 · Rendimiento de lo nuevo

- La lista de reseñas virtualizada, con `keyExtractor` estable y las tres estabilizaciones (`memo`, `useCallback` en `renderItem`, `useCallback` en el handler del padre).
- Imágenes del detalle y de la tarjeta con `expo-image`, medidas en un Android de gama media. El emulador no cuenta.
- Memoria y CPU con el **Profiler de Android Studio** durante un scroll largo con fotos: la curva se aplana, no sube en escalera. La captura va en el PR.
- Los dos hilos, con el monitor de rendimiento: si cae el FPS de JS es nuestro código, si cae el de UI son imágenes, sombras o layout.

Los controles y la capa que los bloquea

Es la tabla que resuelve la mitad de las preguntas del sprint. Se pega en el PR.

Control	Capa que lo bloquea	Qué hace la UI
"Publicar anuncio"	capacidad	oculto; en su sitio, "Hazte proveedor"
"Editar" en anuncio ajeno	propiedad	oculto
"Editar" en anuncio retirado	estado	deshabilitado + motivo
"Reservar" tu propio anuncio	relación	deshabilitado + motivo
"Aceptar" una reserva	propiedad del anuncio	oculto para el cliente
"Cancelar" una reserva completada	estado	deshabilitado + motivo
"Reseñar" ya reseñada	unicidad	deshabilitado + "Ya reseñaste esta reserva"
"Reseñar" fuera de plazo	tiempo	deshabilitado + su propio mensaje
"Reseñar" sin completar	estado	deshabilitado + motivo
Pestaña de moderación	rol de plataforma	oculta
"Moderar" tu propia reseña	no ser el autor	deshabilitado + motivo

Cómo se escribe el código este sprint

Sin documentación. Ni JSDoc, ni cabeceras, ni bloques explicativos, ni comentarios que repiten lo que la línea ya dice. Si un trozo pide explicación, se extrae a una función con nombre. Única excepción: **una** línea de comentario cuando el porqué no es deducible del código y viene de fuera (una rareza del backend, un fallo conocido de una librería).

Y por eso el listón del nombre sube. `canReviewBooking`, `widenRadius`, `problemMessageKey` se leen solos. `handleData`, `utils.ts`, `helper` no entran. El nombre es la única documentación que queda.

Los límites, y se miran en la revisión:

- Un archivo no pasa de ~120 líneas; una función, de ~30; el anidamiento, de 3 niveles.
- Cero `any`. Cero `as` salvo `as const`. Todo lo que entra de la red pasa por `parse`.
- Un `switch` sobre una unión discriminada termina en `assertNever`.
- Identificadores, archivos, ramas, commits y claves de i18n **en inglés**. El texto que ve el usuario vive en `en.json` y `es.json`.
- Nada de estado de servidor en `useState`. Eso es TanStack Query.

El diseño, sin nada de sobra. Cada pantalla tiene los controles que su criterio de aceptación nombra, y ni uno más:

- Se reutilizan `Button`, `Chip`, `TextField` y `StatusBadge`. Un componente nuevo se crea cuando lo usan **dos** pantallas, no antes.
- Ni un color fuera del tema. Ni un `style={{ backgroundColor: '#fff' }}`, ni un `dark:` suelto, ni un `blue-500`. Si falta un token semántico, se añade al tema.

- Sin librería de iconos, sin ilustraciones de estado vacío, sin animaciones. El único feedback táctil es `active:`, y ese sí es obligatorio.
- Sin pantalla de ajustes, sin onboarding, sin perfil: no están en ninguna historia.

Dependencias nuevas, cerradas: `expo-updates`, `expo-image-picker`, `expo-image-manipulator`, `expo-crypto` y `expo-build-properties`. Cualquier otra la aprueban los dos.

Nada entra a `main` sin `verify.sh` en verde, y el PR lo revisa el otro con el teléfono en la mano. Si no puede reproducir un punto, se devuelve.

Criterios de aceptación · QA antes del PR

Jorge

- ☐ Sin la capacidad de proveedor no existe "Mis anuncios" ni "Publicar". Me hago proveedor desde la app y aparecen sin reiniciar.
- ☐ Publico un anuncio en 4 pasos. Elijo "por hora" y me pide horas mínimas; elijo "presupuesto" y no me pide precio en ninguna parte.
- ☐ Salgo de la app a mitad de la publicación, vuelvo, y el borrador está en "Mis anuncios" para retomarlo.
- ☐ Subo dos fotos desde la galería y aparecen en el anuncio publicado. Una foto de 12 MP no tumba la app.
- ☐ Abro el anuncio de otra cuenta y **no hay** botón de editar. Fuerzo el `PATCH` con la app y el servidor lo rechaza con `not_owner`, y la app lo enseña traducido.
- ☐ Pauso un anuncio y el cambio se ve en "Mis anuncios" y en la búsqueda, sin tirar de recarga manual.
- ☐ Con una cuenta de moderador retiro un anuncio denunciado desde la cola; con una cuenta normal la pestaña de moderación no existe.
- ☐ Modero mi propia reseña: el botón está deshabilitado y dice por qué.
- ☐ La app corre en el emulador de Android y en un Android físico contra el backend local, con `10.0.2.2` y con la IP de la red, cada uno en su sitio.
- ☐ Borro `android/`, corro `npm run -w mobile android` y todo vuelve a funcionar. Si algo se pierde, es que estaba editado a mano donde no tocaba.
- ☐ Un compañero instala la app desde el enlace en un **Android físico** y la abre, y la app **carga datos**: el HTTP en claro del perfil de preview está resuelto.
- ☐ Un compañero instala la app desde el enlace en un **iPhone físico** y la abre.
- ☐ Publico un cambio de traducción por OTA y llega sin rebuild.
- ☐ `./scripts/verify.sh` en verde, y en CI también.

Salvador

- ☐ Comparto un anuncio por WhatsApp desde el detalle; el enlace abre ese anuncio en la app, incluso con la app cerrada.
- ☐ En el detalle, el precio se lee `$450 / hora · mínimo 2 h` y la valoración `4,8 · 200 reseñas`, con plural correcto en 0, 1 y 2.
- ☐ Doy a favorito y el corazón cambia **al instante**. Con la red caída se revierte solo y la lista de búsqueda queda coherente.
- ☐ Pulso "Reservar" dos veces seguidas y se crea **una** reserva. Se comprueba en la pestaña de red, no a ojo.

- ☐ Vuelvo atrás tras reservar y el estado nuevo ya está ahí.
- ☐ Pongo el avión a mitad de una reserva: sale un mensaje claro y no pierdo lo escrito.
- ☐ Reseño una reserva completada. Vuelvo a entrar y el botón está **deshabilitado** con "Ya reseñaste esta reserva". No desaparece.
- ☐ Con una reserva completada hace 40 días, el mensaje es el de plazo cerrado, distinto del anterior.
- ☐ Intento reseñar una reserva que no es mía y una que no está completada: cada una con su motivo.
- ☐ Los tests de `canReviewBooking` cubren los cuatro motivos, el camino feliz y el borde de los 30 días, sin tocar el reloj del sistema.
- ☐ Los cuatro estados existen en reservas y en reseñas: skeleton con forma, error en lenguaje llano, vacío inicial y vacío por filtro.
- ☐ Con el teléfono en alemán y la fuente al 200%, ni el formulario de reserva ni el de reseña cortan texto.
- ☐ Con VoiceOver, una reserva es **una** parada del lector, no siete.
- ☐ Al enviar un formulario vacío, el lector anuncia el primer error.
- ☐ La lista de reseñas con 500 elementos no baja de 55 FPS en el Android de gama media.
- ☐ Captura del Profiler de Android Studio en el PR: memoria plana en un scroll largo con fotos, sin escalera.

Definition of Done · quién responde de cada línea

Bloque	Criterio	Responde
Funcional	Cuatro estados en búsqueda	cerrado en sprint 1
Funcional	Vacío por filtro ofrece ampliar radio o limpiar	cerrado en sprint 1
Funcional	Botón deshabilitado mientras la petición vuela	los dos
Funcional	Volver atrás tras una mutación muestra el estado nuevo	Salvador
Permisos	Un proveedor no ve editar en el anuncio de otro	Jorge
Permisos	Reseñar dos veces muestra el motivo, no un botón ausente	Salvador
Permisos	Reseñar fuera de plazo tiene su propio mensaje	Salvador
Permisos	Denegar ubicación ofrece selector de ciudad	cerrado en sprint 1
Dinero e i18n	El mismo precio en es-MX, en-US y de-DE	cerrado en sprint 1
Dinero e i18n	Modelo de precio visible: "/hora · mínimo 2 h"	Salvador
Dinero e i18n	Distancia en km o millas según locale	cerrado en sprint 1
Dinero e i18n	Plurales correctos con 0, 1 y 2 reseñas	Salvador
Dinero e i18n	Layout intacto en alemán	Salvador
Accesibilidad	Una tarjeta = una parada del lector	Salvador
Accesibilidad	Errores anunciados al enviar	Salvador
Accesibilidad	Fuente al 200% sin cortes	Salvador
Accesibilidad	Ningún estado comunicado solo por color	los dos

Bloque	Criterio	Responde
Accesibilidad	Área táctil $\geq 44 \times 44$	los dos
Rendimiento	55 FPS con 5.000 tarjetas	cerrado en sprint 1
Rendimiento	Memoria estable en scroll largo	Salvador
Rendimiento	Modo avión a mitad de una reserva	Salvador
Entrega	Probado en dispositivo físico	los dos
Entrega	Instalado desde preview por alguien que no lo escribió	Jorge
Entrega	<code>verify.sh</code> en verde y <code>main</code> protegida	Jorge

Calendario

Día	Jorge	Salvador
0	Postman, Android Studio y SDK, credenciales de tienda	Postman, Android Studio y SDK, motivos de error
1– 2	J0 · kit compartido a <code>main</code>	S1 · detalle, favorito, compartir
3	J6 · primera build de preview instalable	S1 · cierre
4– 5	J1 y J2 · proveedor y mis anuncios	S2 · reservar
5	Demo cruzada interna. Se revisa el delta del backend	Demo cruzada interna
6– 8	J3 · publicar en 4 pasos	S3 y S4 · reservas y la política de reseña
9	J4 y J5 · editar y moderación	S5 · i18n y accesibilidad
10	J6 · build final, OTA, README	S6 · rendimiento
10	Congelación: solo QA del DoD en dispositivo físico. Ni una función nueva.	Igual

El día 3 hay build instalable aunque la app haga poco. Es lo que evita descubrir el problema de firma de iOS el último día.

Fuera de alcance

No se trabaja en esto aunque sobre tiempo:

- Chat en tiempo real (US-11) y pagos (US-12). Están fuera por el enunciado.
- Generar tipos desde el OpenAPI del backend. Decidido el día cero, anotado en el delta.
- Pantalla de perfil, ajustes, onboarding, notificaciones push, modo oscuro manual, búsqueda por mapa, historial de búsquedas. Ninguna aparece en una historia.
- Cualquier comprobación de permiso que pretenda proteger algo desde el cliente.

- Un backend propio. La API existe y se consume.

Si sobra tiempo, va a **profundidad**: más estados cubiertos, mejor comportamiento con red mala, accesibilidad más fina, y un `verify.sh` más rápido. No a funciones nuevas.